·sarial Search Algorithms in Artificial Intelligence (AI)

ted : 23 Jul, 2025

arial search algorithms are the backbone of strategic decision-making in artificial intelligence, it enabl :o navigate competitive scenarios effectively. This article offers concise yet comprehensive advantage .gorithms from their foundational principles to practical applications. Let's uncover the strategies that ent gameplay in adversarial environments.

of Content

hat is an Adversarial search?
versarial search algorithms
nimax algorithm
pha-beta pruning
versarial search algorithm Implementations using Connect-4 Game
plications of adversarial search algorithms
nclusion
Q's on Adversarial search algorithms

is an Adversarial search?

versarial search is a well-suited approach in a competitive environment, where two or more agents ha ng goals. The adversarial search can be employed in two-player zero-sum games which means what r one player will be the misfortune for the other. In such a case, there is **no win-win outcome**. In artifi ence, adversarial search plays a vital role in decision-making, particularly in competitive environments :ed with games and strategic interactions. By employing adversarial search, AI agents can make optin is while anticipating the actions of an opponent with their opposing objectives. It aims to establish an e decision for a player by considering the possible moves and the counter-moves of the opponents.

versarial search in competitive environments can be utilized in the below scenarios where the AI syste determining the best course of action by both considering the possible moves and counter-moves of nts.

agent seeks to boost their utility or minimize their loss.
agent's action impacts the outcomes and objectives of the other agents.
tionally, strategic uncertainty arises when the agents may lack sufficient information about each othe egies.

Adversarial Search in AI

e-playing: The Adversarial search finds a significant application in game-playing scenarios, including wned games like chess, Go, and poker. The adversarial search offers the simplified nature of these ga

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

represents the state of a game in a straightforward approach and the agents are limited to a small nu
tions whose effects are governed by precise rules.
sion-making: Decision-making plays a central role in adversarial search algorithms, where the goal is
best possible move or strategy for a player in a competitive environment against one or more compone
requires strategic thinking, evaluation of potential outcomes, and adaptive decision-making througho
e.

sarial search algorithms

rch algorithms like DFS, BFS, and A* can be well-suited for single-agent environments where there is
ompetition or conflict between multiple agents. These algorithms are suitable for finding an optimal s
scenarios. On the other hand, in **zero-sum games** where two players compete directly against each o
rial search algorithms like **Minmax** and **Alpha-Beta pruning** are more appropriate since these algori
ermine the best course of action for each player in zero-sum games.

ax algorithm

imax algorithm is claimed to be a recursive or backtracking algorithm that is responsible for choosing
timal move in the conflicting environment. The Minimax algorithm operates on a tree structure knowr
ee, which is the collection of all the possible moves in the corresponding game states in a given game
ee's leaf node accommodates all the possible moves. The game state denotes the current board cond
very single move, the game state changes and the game tree gets updated height-wise. When visuali
he tree often resembles an inverted tree, with the root representing the current game state and the br
nting possible moves.

orithm simply works by proceeding down to the tree until it reaches the leaf node and then it backs u
x values through the tree as the recursion unwinds. The prime motive of the Minimax algorithm is to
ze the chance of winning for the maximizer. In the game tree, every node is assigned weights that infl
nces of winning, the higher the weights, the higher the chances of winning.

minologies in the Minimax algorithm

max Tree: A tree structure shows all possible moves and game states in a game, used by Minimax to
move.
(Maximizer) - Maximizer seeks favorable outcomes in games by maximizing chances of winning for
selves as players.
(Minimizer) - Minimizer reduces winning chances, making moves leading to least favorable outcome f
mizer in games.
l state - Starting state of game board, representing the configuration at the beginning of the game.
inal state -Terminal states are the final outcomes of a game board, resulting in either a win, loss, or d
istic Evaluation Function: Function evaluates game states for maximizer, assigning numerical values
e state based on piece positions, material advantage, and board control.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

inimax search algorithm help in finding the optimal strategy?

inal state: The Minimax value of each state in the game tree can be used to determine the optimal st
MINIMAX(s) refers to the utility of being in that state that assumes both players play optimally from t
nd of the game.
terminal state: In the case of the non-terminal state, when MAX's turn arrives at the game, MAX aim
e which leads to a state of maximum value, conversely MIN prefers to choose the move that leads to a
nimum value. It can be formulated as below:

$$NIMAX(S) = \begin{cases} UTILITY(s, \text{MAX}) \rightarrow \text{if } IS_TERMINAL(s), \\ \max_{a \in Actions(s)} \ MINIMAX(RESULT(s,a)) \rightarrow \text{if } TO_MOVE(s) = \\ \min_{a \in Actions(s)} \ MINIMAX(RESULT(s,a)) \rightarrow \text{if } TO_MOVE(s) = \end{cases}$$

ocode for Minimax algorithm

eudocode for the Minimax algorithm is given below:

MAX(node, depth, maximizingPlayer): This function recursively evaluates the minimax value of each
e game tree. It alternated between maximizing and minimizing player nodes based on the maximizing
meter

```
IAX(node, depth, maximizingPlayer):
.f depth is 0 or node is a terminal node:
    return the heuristic value of node
.f maximizingPlayer:
    bestValue = -INFINITY
    for each child of node:
        value = MINIMAX(child, depth - 1, FALSE)
        bestValue = max(bestValue, value)
    return bestValue
else:
    bestValue = +INFINITY
    for each child of node:
        value = MINIMAX(child, depth - 1, TRUE)
        bestValue = min(bestValue, value)
    return bestValue
```

BestMove(board): This function is responsible for iterating through all possible moves on the board a
uates each moves using minimax algorithm. Then it returns the moves with the highest minimax value
mizing player.

```
BestMove(board):
bestValue = -INFINITY
```

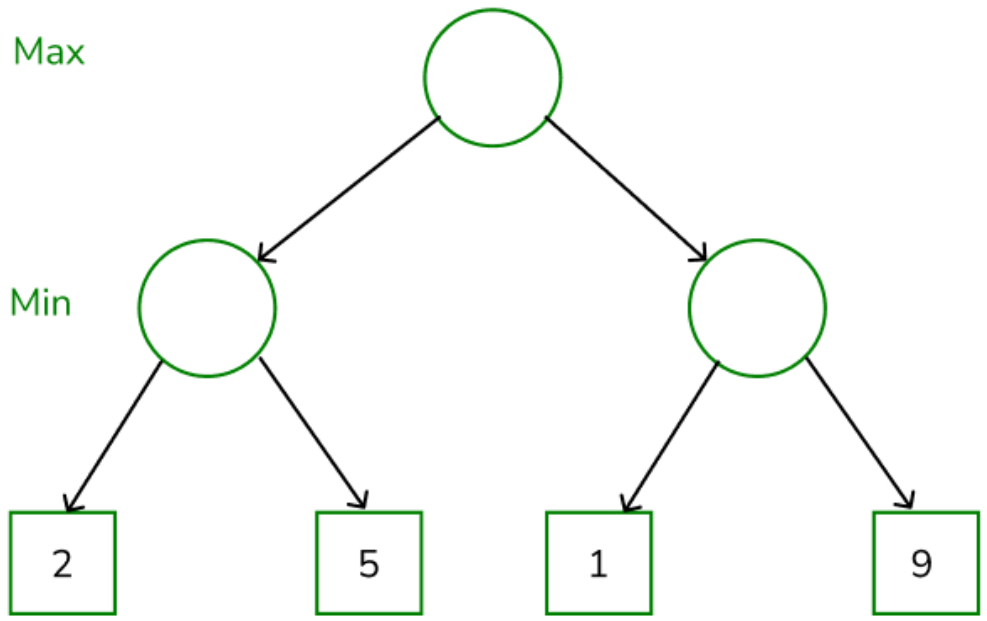
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
estMove = NULL
or each possible move in board:
    if move is valid:
        make the move on the board
        value = MINIMAX(board, depth, FALSE)
        undo the move on the board
        if value > bestValue:
            bestValue = value
            bestMove = move
eturn bestMove
```

tanding the Minimax algorithm: Decision-making in game trees

imax algorithm is a **backtracking algorithm** that **recursively** tries different possibilities and backtrack tion is not found. In the context of the Minimax algorithm, this involves exploring all possible moves ee that evaluate each potential outcome and making decisions based on the evaluations.

gram below represents the binary tree that holds the possible game states and moves in a game, wit le representing a final game state.



figure(a) Example of Minimax algorithm that holds the possible game states

zer and Minimizer

linimax algorithm, there are two players such as the maximizer and the minimizer. The maximizer aim ze its score (e.g., by winning the game), conversely, the minimizer aims to minimize the maximizer's sc preventing the maximizer from winning).

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

e of decision-making

l decision-making: An example scenario in Figure (a) within the perfect binary tree causes the maxim
e the first move at the root. The maximizer has the option to choose to traverse through the tree, in th
eds to choose the path between left or right.

nizer's choice: Assuming that maximizer traverses to the left where it reaches the minimizer node, th
nizer has the responsibility of selecting the optimal value and it chooses between two values (2 and
ctive of the minimizer is to minimize the maximizer's score, so it chooses the lowest value 2. The value
passed to the root node so that it can be compared later when the player earns an optimal value whe
rses to the right side of the game tree.

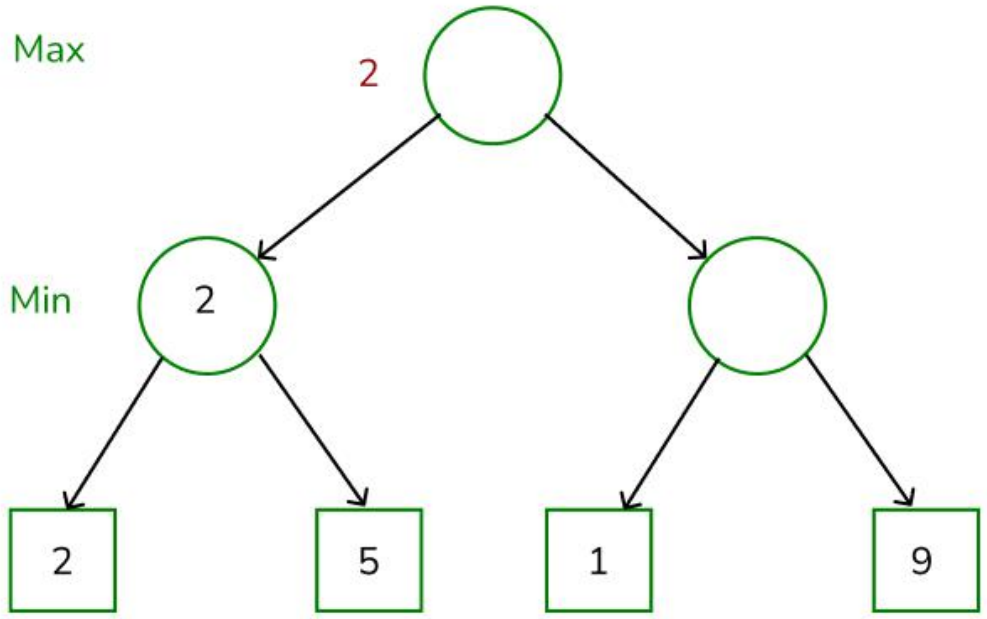


Figure (b) Minimizer's choice

ersal to the Right: The maximizer then traverses to the right side of the game tree and encounters an
f decision nodes. Eventually, it reaches another minimizer node where the minimizer again selects the
nal minimal value 1 among its options.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

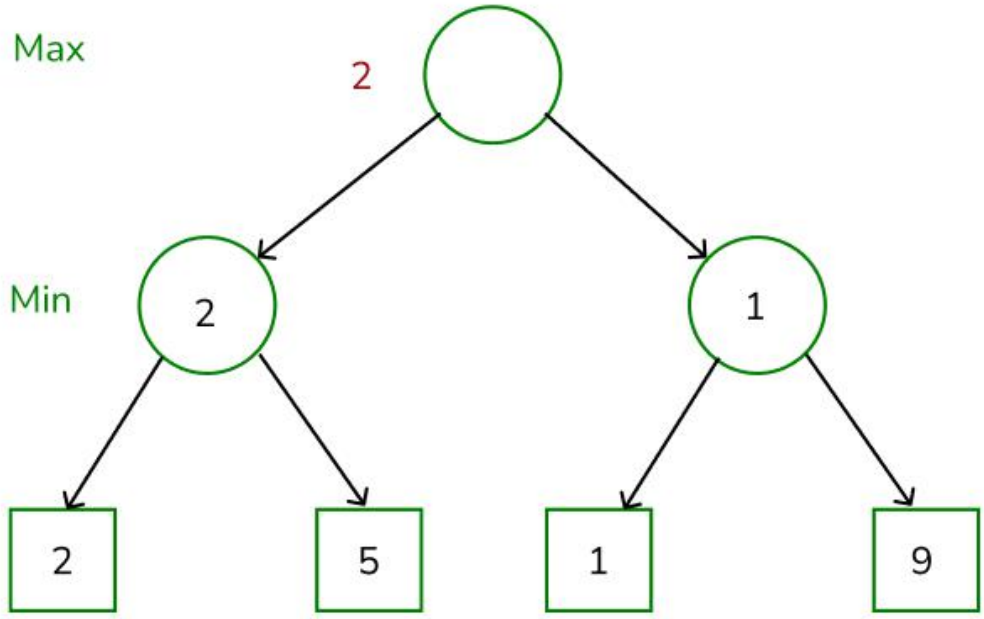
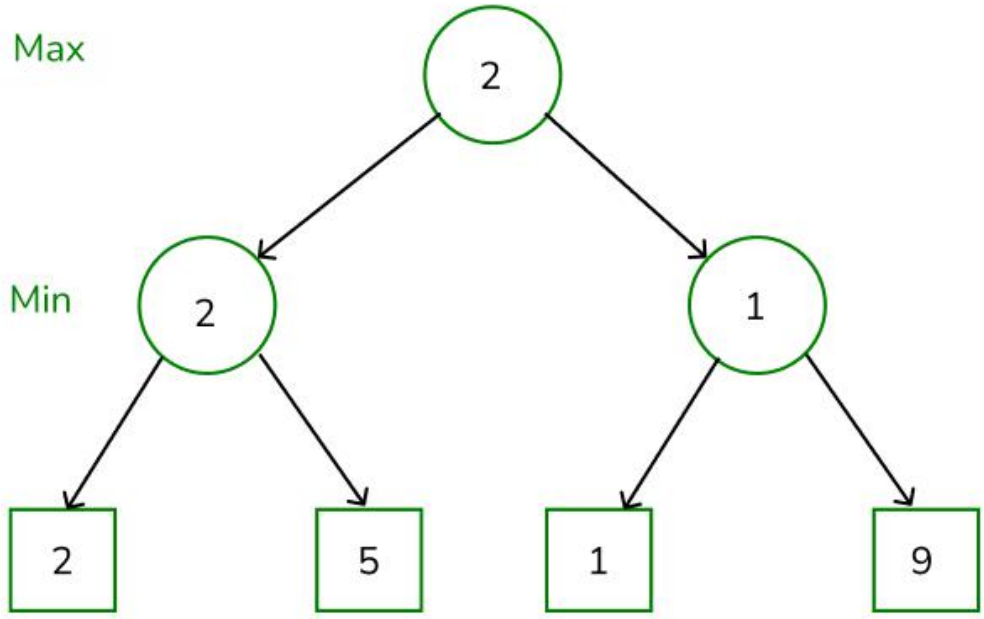


Figure (c) The maximizer traversing through the right

rsive process: The process continues recursively where each player makes decisions based on wheth
naximizer or minimizer and the objective of maximizing or minimizing the score respectively.
parison and outcome: The maximizer node (root node) already holds the value of 2, when the player
ner optimal value that is 1 while traversing in the right path, it is then compared to get the final optim
e. The maximizer earns the optimal value 2 based on the decisions made by both players.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Final outcome of binary game tree

nd Space complexity of Minimax algorithm

omplexity

imax algorithm explores the complete game tree using a depth-first search approach that considers a
e moves at each level. The time complexity of the minimax algorithm can be written as,

$$O(b^m)$$

enotes the legal moves available at each point,
enotes the maximum depth of the tree.

nplexity arises because, at each level of the tree, the minimax algorithm must consider all b legal mo
cess repeats recursively for m levels until a terminal state is reached.

omplexity

ce complexity of the minimax algorithm relies on how the algorithm generates and stores the actions
here if all the possible actions are generated and stored simultaneously, the space complexity can be

$$O(bm)$$

enotes the legal moves available at each point,

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

enotes the maximum depth of the tree.

ses because the algorithm needs to store all the possible actions at each level of the game tree whic
nential growth in space requirements with the depth of the tree.

er, if the actions are generated and processed one at a time, the space complexity reduces to O(m).

-beta pruning

beta pruning is an optimization technique for a minimax algorithm. It reduces computation time by a h
llowing the user to traverse faster and deeper into the tree. It stops evaluating when at least one pos
n found that typically proves the move to be worse than the previously examined move. The minimax
l on depth-first search which considers the nodes along a single path in a tree. But Alph-Beta pruning
o major parameters in MAX-VALUE(state, alpha, beta), representing Alpha plays a maximizer role, wh
ays a minimizer role.

a - denotes the value of the best or highest value.
- denotes the value of the best or lowest value.

lpha-beta pruning is considered as an effective algorithm in the adversarial search?

ch elimination: The Alpha-beta pruning algorithm relies on the branches of the game tree that can b
nated which promotes the more promising subtree by limiting the search time.
ch and bound: Alpha-beta pruning is based on the branch and bound method which offers a way to s
ptimization problem by dividing them into sub-problems. It allows the algorithm to effectively remov
ches that did not yield the optimal solution.
istic improvements: To limit the computation time, the heuristic Alpha-beta tree search cuts off the s
and applies the heuristic evaluation function to states that treat the non-terminal nodes as if they we
inal nodes.

code for Alpha-beta pruning

eudocode for Alpha-beta pruning is given below

```
ion ALPHA-BETA(node, depth, alpha, beta, maximizingPlayer):
f depth == 0 or node is a terminal node:
    return the heuristic value of node
f maximizingPlayer:
    for each child of node:
        alpha = max(alpha, ALPHA-BETA(child, depth - 1, alpha, beta, FALSE))
        if beta <= alpha:
            break   // Beta cut-off
    return alpha
lse:
    for each child of node:
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        beta = min(beta, ALPHA-BETA(child, depth - 1, alpha, beta, TRUE))
        if beta <= alpha:
            break   // Alpha cut-off
    return beta
```

tanding the Alpha-beta pruning: Decision-making in game trees

ees can hold abounding branches that can lead to different game states and implementing alpha-bet in such game trees can help to prune away the branches if there already exists a better move. This m rithm computationally efficient since there is no need to search the other branches if there already ex ove in the game tree.

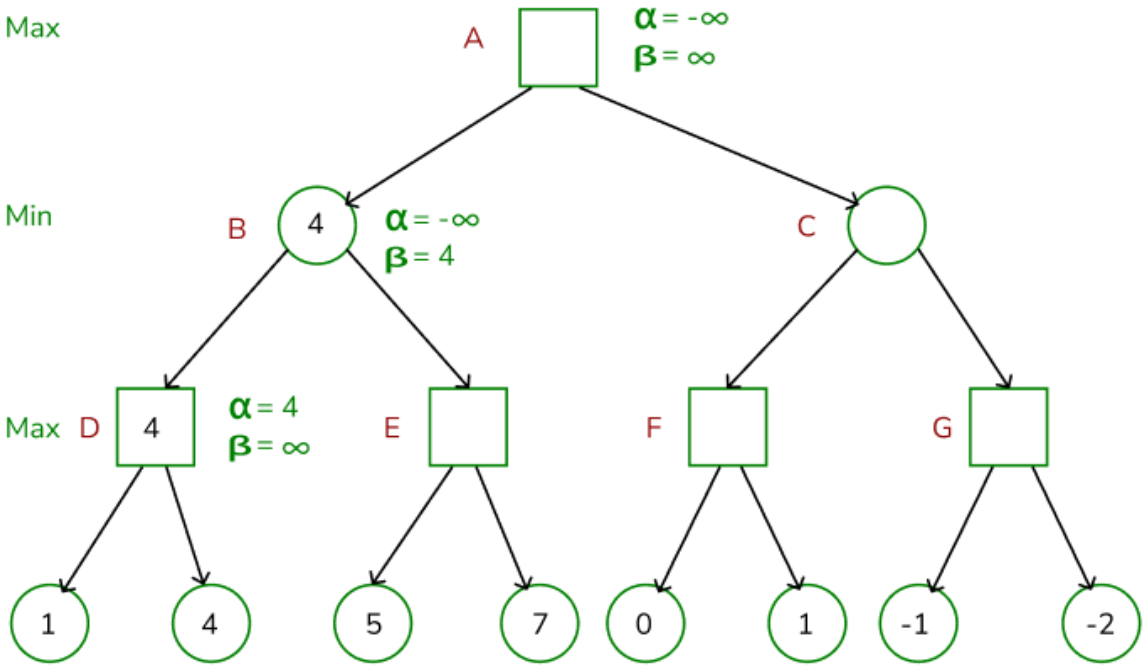
ode in the game tree, the maximizing player is used to store the maximum score in the alpha which a max node contains only the maximum scores. Similarly, the minimizing player is assured of a minimu hich is stored by the beta. These two values of alpha and beta are updated and maintained by the m.

ook at the below example, initially both players begin with the worst possible values they can score, e the alpha is assigned with the value of minus infinity, whereas the beta is assigned with the value o We have a condition to prune the branches, if the alpha is greater than or equal to the beta value the e the branch.

pha is the maximizer and beta is the minimizer.

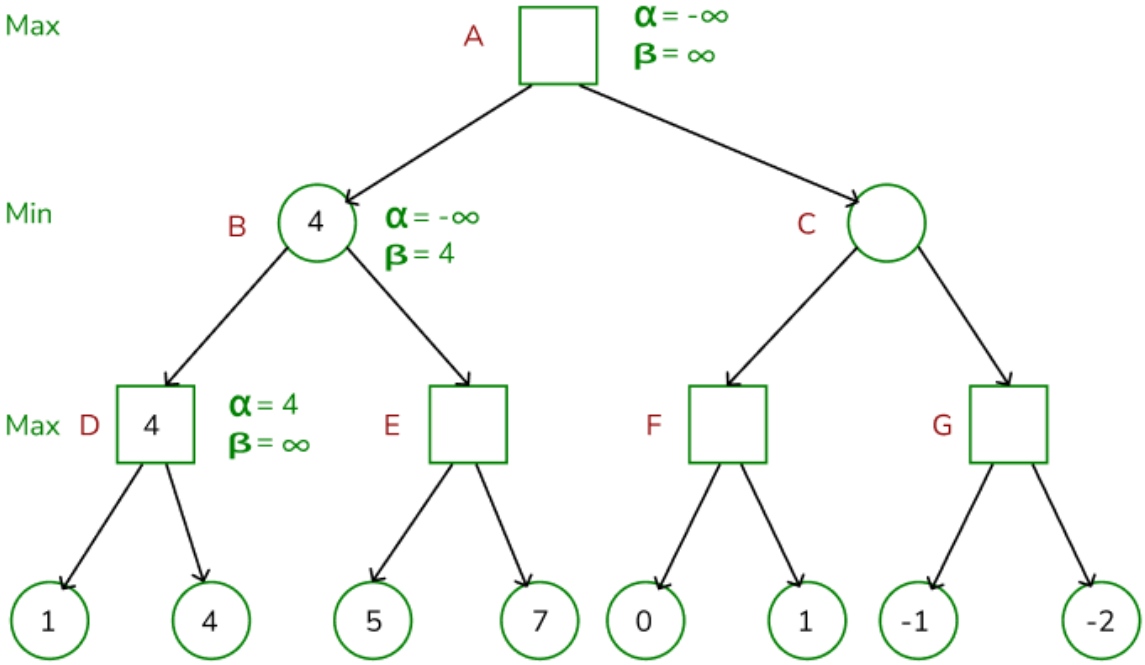
the maximizer node A sets alpha to negative infinity and beta to positive infinity. Assuming that node node B which inherits these values. Node B which is also a maximizer picks node D. At D, the maximi the max value (4) and then updates its alpha. Upon returning to B (minimizer), it compares its curren alpha value. If D's alpha is smaller, B's beta value gets updated. In our example, B's beta gets updat s alpha value is smaller.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Initial game state

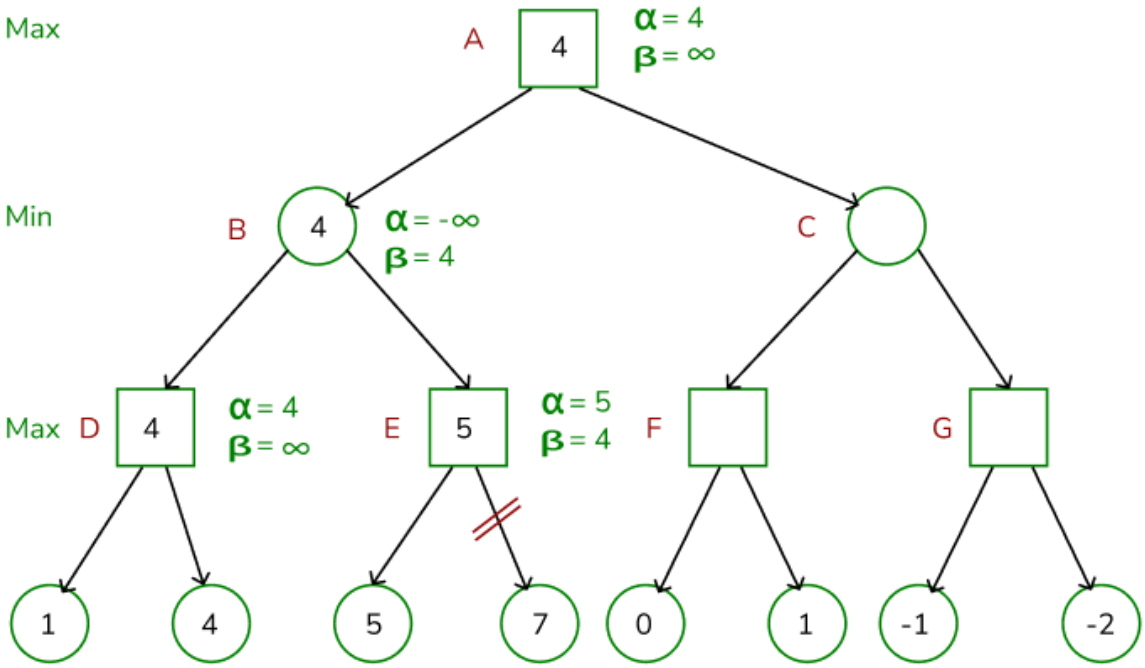
ode B selects node E and passes down its current alpha and beta values to node E. Node E inherits th negative infinity and beta value of 4. As a maximizer, E explores its left leaf node that holds a value is greater than negative infinity, E's alpha gets updated. Given that alpha is greater than beta, the oth E is unnecessary to explore so it gets pruned.



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

values of B and D were found

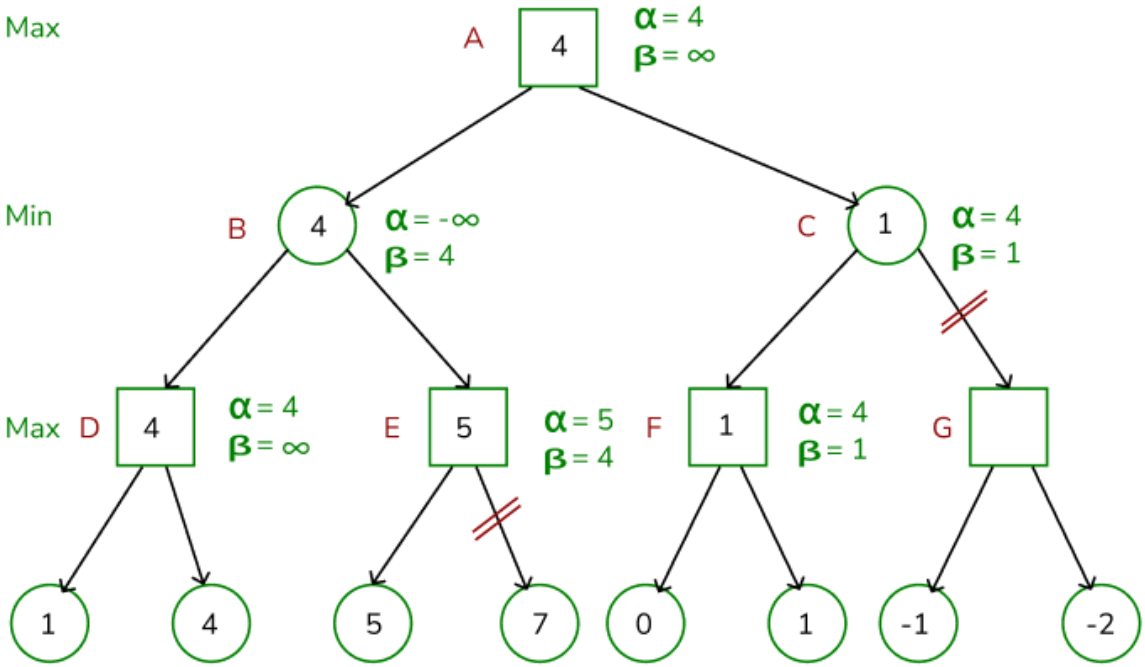
: B, it compares its beta value with node E's alpha, if E's alpha value is lesser than B's beta updates w
However, if E's alpha is greater than B's beta, B's beta remains unchanged. In our example, since B's b
ater than E's alpha, so B's beta value remains the same. Returning to node A, its alpha value updates
value because B's beta is greater than A's alpha.



The maximizer yields the value 4

: A, the maximizer's alpha becomes 4 ensuring a value of 4 or higher. A then explores its right child w
At C, alpha is 4 and beta is positive infinity. C then chooses its left child F. Node F inherits the alpha
a value positive infinity from the parent node. F's left child yields 0 updating the F's alpha to 4 and al
ild yields 1. Although F can achieve 1, alpha remains 4. C receives 1 from F which makes the beta 1.
greater than beta, the entire subtree of node G gets pruned. This pruning logic ensures that, as C is
eed 1 or less, it's irrational for A to choose C over B, where A is already guaranteed 4.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



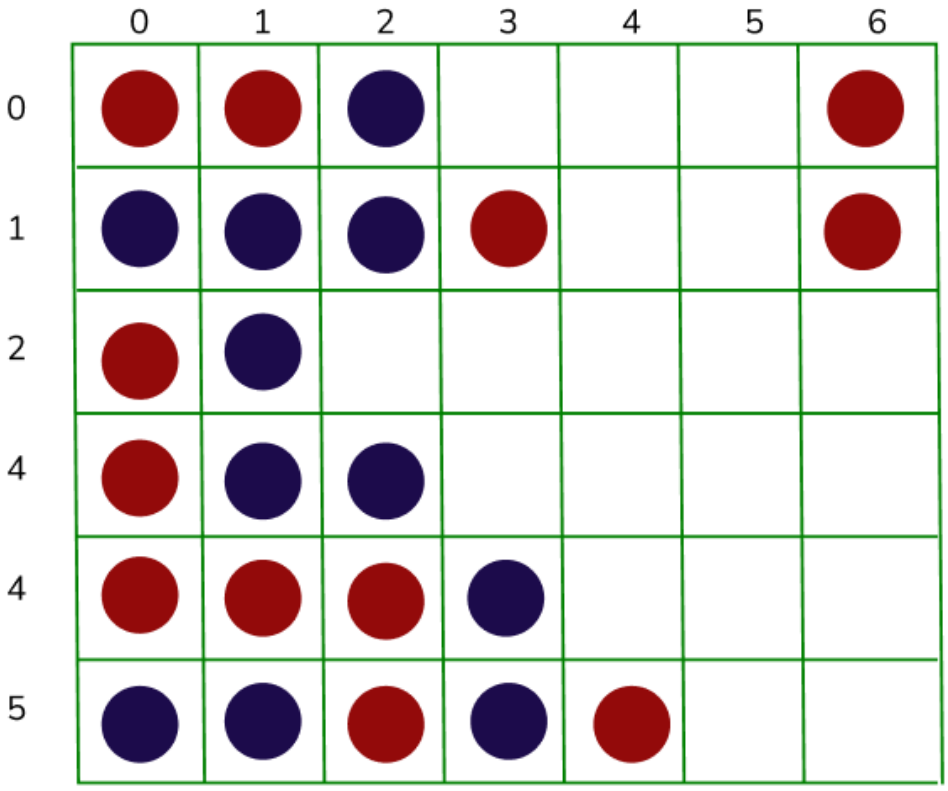
The Maximizer still holds the max value 4 and branch G gets pruned.

'sarial search algorithm Implementations using Connect-4 Game

t-4 is a game played between two players who take turns dropping discs of their chosen color into a .y suspended grid with seven columns and six rows. The objective is to be the first to form a line of fo scs either horizontally, vertically, or diagonally is considered a win. The game is considered solved wh an always win or is forced to draw. This implies that there exists an optimal strategy for both players g that the outcome of the game can be determined in advance. The Connect-4 game is said to be a ze ecause the advantage of one player will be the disadvantage of its opponent.

xample, we will be using the Minimax algorithm to find the optimal strategy in the connect-4 game. T the below illustration, The player X chose the color red and the player O chose the color blue. In the b ment, it's now player O's turn to make a move. We will implement a minimax algorithm to find the op for player O.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Connect-4 game where the player O should make the optimal move to win the game

ıentation of Connect-4 using Minimax algorithm

Check if there are any moves left

ɜs_left: This function checks if there are any empty cells left on the game board. If there are any empt
ɜturns the value True which indicates that there are moves left to be made. Otherwise, it returns False
ıg that the game board is full and no moves can be made.

```
_moves_left(board):
r row in board:
  for cell in row:
      if cell == '':
          return True
turn False
```

Evaluate the board

e: We define this function to evaluate the current state of the game board and it returns a score base
r there is a winning configuration for a player O such as horizontally, vertically, or diagonally. If there
ı configuration it returns a score of 10, otherwise it returns 0.

```
aluate(b):
r row in range(6):
  for col in range(4):
      if b[row][col] == b[row][col+1] == b[row][col+2] == b[row][col+3] == 'o':
          return 10
  for col in range(7):
      for row in range(3):
          if b[row][col] == b[row+1][col] == b[row+2][col] == b[row+3][col] == 'o':
              return 10
  for row in range(3):
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        for col in range(4):
            if b[row][col] == b[row+1][col+1] == b[row+2][col+2] == b[row+3][col+3] == 'o':
                return 10
    for row in range(3, 6):
        for col in range(3):
            if b[row][col] == b[row - 1][col+1] == b[row-2][col+2] == b[row - 3][col+3] == 'o':
                return 10
    return 0
```

Implement the Minimax algorithm

x: We define the minimax function to implement the minimax algorithm that makes decision-making s
player games. It recursively evaluates the possible moves and returns the best score for the current p

```
nimax(board, depth, is_max):
ore = evaluate(board)

 score == 10:
  return score - depth
 not is_moves_left(board):
  return 0
 is_max:
  best_value = -float('inf')
  for col in range(7):
      for row in range(5, -1, -1):
          if board[row][col] == '':
              board[row][col] = 'x'
              best_val = max(best_val, minimax(
                  board, depth+1, not is_max))
              board[row][col] = ''
              break
  return best_val
se:
  best_value = float('inf')
  for col in range(7):
      for row in range(5, -1, -1):
          if board[row][col] == '':
              board[row][col] = 'o'
              best_val = max(best_val, minimax(
                  board, depth+1, not is_max))
              board[row][col] = ''
              break
  return best_val
```

Find the optimal move for player O

timal_move: We define this function to discover the optimal move for player O by simulating all poss
and evaluating their scores using the minimax algorithm. It returns the coordinates of the best move.

```
nd_optimal_move(board):
st_move = None
st_val = -float('inf')

r col in range(7):
  for row in range(5, -1, -1):
      if board[row][col] == '':
          board[row][col] = 'o'
          move_val = minimax(board, 0, False)
          board[row][col] = ''

          if move_val > best_val:
              best_val = move_val
              best_move = (row, col)
          break
turn best_move
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Test the given board configuration

e will test the given board configuration and print out the optimal move for player O to win a game. I
ssible winning move, it prints a message indicating that player O cannot win with the current board
ration.

```
= [
x', 'x', 'o', '', '', '', 'x'],
o', 'o', 'o', 'x', '', '', 'x'],
x', 'o', '', '', '', '', ''],
x', 'o', 'o', '', '', '', ''],
x', 'x', 'x', 'o', '', '', ''],
o', 'o', 'x', 'o', 'x', '', '']

l_move = find_optimal_move(board)
imal_move:
int("The optimal move for player O to win is:", optimal_move)

int("Player O cannot win with the current board confguration")
```

ete code

```
s_moves_left(board):
or row in board:
    for cell in row:
        if cell == '':
            return True
eturn False


valuate(b):
or row in range(6):
    for col in range(4):
        if b[row][col] == b[row][col+1] == b[row][col+2] == b[row][col+3] == 'o':
            return 10
    for col in range(7):
        for row in range(3):
            if b[row][col] == b[row+1][col] == b[row+2][col] == b[row+3][col] == 'o':
                return 10
    for row in range(3):
        for col in range(4):
            if b[row][col] == b[row+1][col+1] == b[row+2][col+2] == b[row+3][col+3] == '(
                return 10
    for row in range(3, 6):
        for col in range(3):
            if b[row][col] == b[row - 1][col+1] == b[row-2][col+2] == b[row - 3][col+3] =

                return 10

    return 0
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
inimax(board, depth, is_max):
core = evaluate(board)

f score == 10:
   return score - depth
f not is_moves_left(board):
   return 0
f is_max:
   best_value = -float('inf')
   for col in range(7):
       for row in range(5, -1, -1):
           if board[row][col] == '':
               board[row][col] = 'x'
               best_val = max(best_val, minimax(
                   board, depth+1, not is_max))
               board[row][col] = ''
               break
   return best_val
lse:
   best_value = float('inf')
   for col in range(7):
       for row in range(5, -1, -1):
           if board[row][col] == '':
               board[row][col] = 'o'
               best_val = max(best_val, minimax(
                   board, depth+1, not is_max))
               board[row][col] = ''
               break
   return best_val


ind_optimal_move(board):
est_move = None
est_val = -float('inf')

or col in range(7):
   for row in range(5, -1, -1):
       if board[row][col] == '':
           board[row][col] = 'o'
           move_val = minimax(board, 0, False)
           board[row][col] = ''

           if move_val > best_val:
               best_val = move_val
               best_move = (row, col)
           break
eturn best_move


 = [
'x', 'x', 'o', '', '', '', 'x'],
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

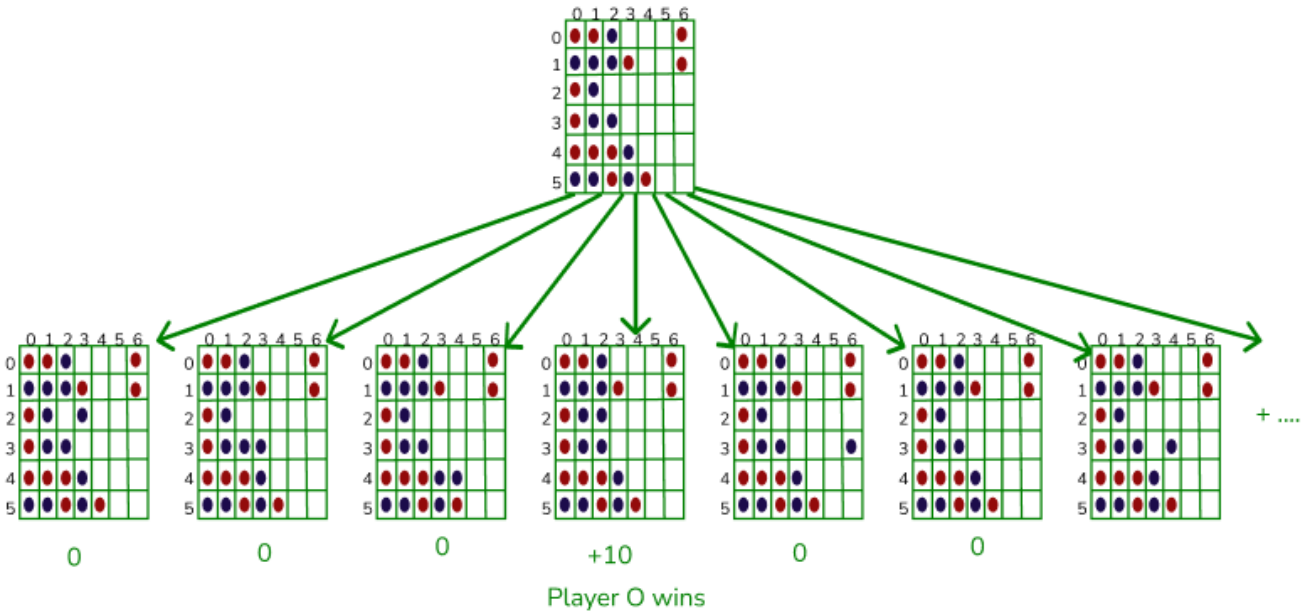
```
'o', 'o', 'o', 'x', '', '', 'x'],
'x', 'o', '', '', '', '', ''],
'x', 'o', 'o', '', '', '', ''],
'x', 'x', 'x', 'o', '', '', ''],
'o', 'o', 'x', 'o', 'x', '', '']

al_move = find_optimal_move(board)
timal_move:
rint("The optimal move for player O to win is:", optimal_move)

rint("Player O cannot win with the current board confguration")
```

```
ptimal move for player O to win is: (2,2)
```

Explanation



Output of Connect-4 where the player O wins

ook at the above output illustration, the player O won when he chose the optimal move such as (2,2).
nt to note that there are multiple possible moves that player O could have made to potentially win th
however, in the given illustration only seven states of the game board are shown to demonstrate the
ıg the optimal move using the minimax algorithm.

cations of adversarial search algorithms

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

d games: Adversarial search is most widely used in various board games like Chess, Checkers, Go an ect Four. The above-explained algorithms can help the computers to play against human opponents computer players.

e Theory: Adversarial search forms the basis of game theory, which is used in various fields like econ cal science, and biology to model strategic interactions between rational decision-makers.

le-solving: Adversarial search algorithms can be used to solve puzzles and optimization problems wh oal is to find the best sequence of moves or actions to achieve a desired outcome.

usion

arial search algorithms have emerged as a powerful tool with diverse applications across numerous d astering complex board games to enhancing cybersecurity, robotics, and automated negotiation syste gorithms facilitate strategic decision-making in dynamic competitive environments.

gs: Blogathon Artificial Intelligence AI-ML-DS

e

tion to AI

pts

Learning in AI

and AI

ve AI

ce

Do Not Sell or Share My Personal Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF